



Fuyun: Bridging the Semantic Gap in Serverless Resource Provisioning via LLM Agents

Qingwen Li^{*}
Zhengyu Lei^{*}
Xiaofang Zhao^{*}

Kai Lv^{*}
Cunchi Lv^{*}

Mingxuan Yang^{*}
Xiao Shi^{*^{*}}

^{*}Corresponding author.

^{*} ICT, CAS; UCAS Beijing, China

AgenticOS 2026

王鑫

2026.04.13



作者介绍
研究背景与动机
系统设计
实验评估
总结与讨论



作者介绍

研究背景与动机

系统设计

实验评估

总结与讨论

作者介绍

史骁

云计算、无服务器计算、算力网

期刊文章:

- [1] 郑晓辉, 史骁, 唐宏伟等. 面向用户体验的动态负载均衡算法研究. 高技术通讯, (31), 4, 359-366, 2021.
- [2] 史骁, 郑晓辉, 唐宏伟等. 面向强一致性的分布式对象存储的 I/O 并行性优化. 高技术通讯, (2), 30, 109-119, 2020.
- [3] 宋永浩, 史骁, 胡斌等. 基于多目标贪心策略的增益最大化团队构建算法. 高技术通讯, (28), 4, 279-290, 2018.

会议文章:

- [1] Cunchi Lv, Xiao Shi, Zhengyu Lei, et al. Dilu: Enabling GPU Resourcing-on-Demand for Serverless DL Serving via Introspective Elasticity. ACM ASPLOS 2025.
- [2] Cunchi Lv, Xiao Shi, Dong Liang, et al. SpecInF: Exploiting Idle GPU Resources in Distributed DL Training via Speculative Inference Filling. NPC 2024.
- [3] Zhengyu Lei, Xiao Shi, Cunchi Lv, et al. Chitu: Accelerating Serverless Workflows with Asynchronous State Replication Pipelines. ACM SoCC 2023.
- [4] Yuxin Yuan, Xiao Shi, Zhengyu Lei, et al. SMPI: Scalable Serverless MPI Computing. IEEE IPCCC 2022.
- [5] Wenting Tan, Xiao Shi, Zhengyu Lei, et al. TrainFlow: A Lightweight, Programmable ML Training Framework via Serverless Paradigm. NPC 2022.





作者介绍

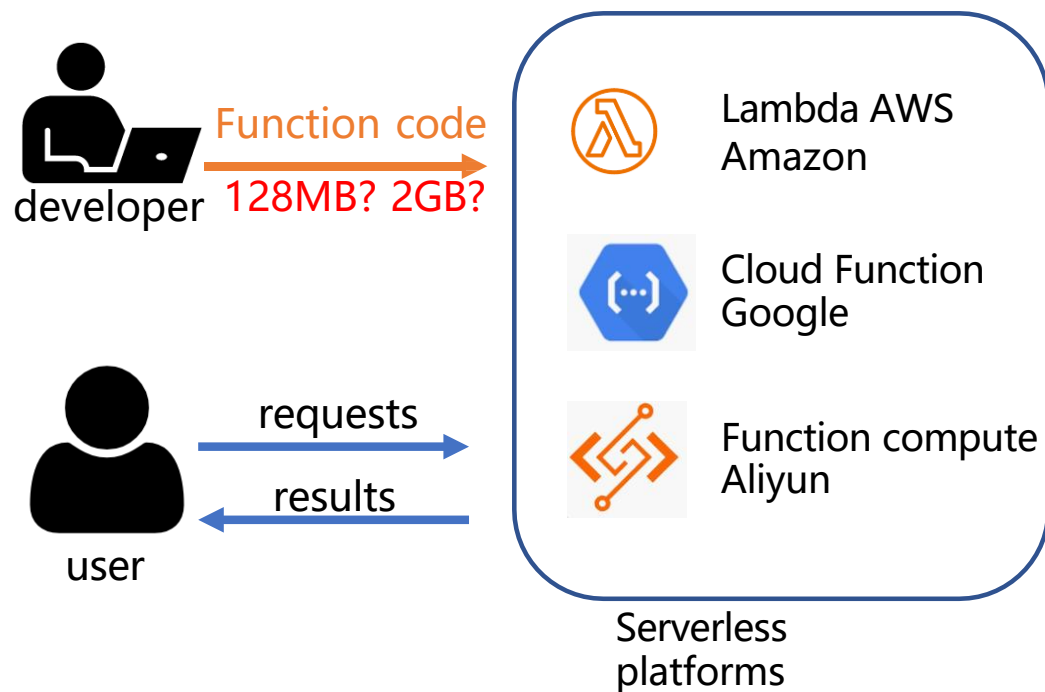
研究背景与动机

系统设计

实验评估

总结与讨论

研究背景与动机



无服务器计算 (Serverless Computing)

无服务器计算可以免去用户的基础设施管理工作，让用户只需聚焦于将应用逻辑封装为无服务器函数。

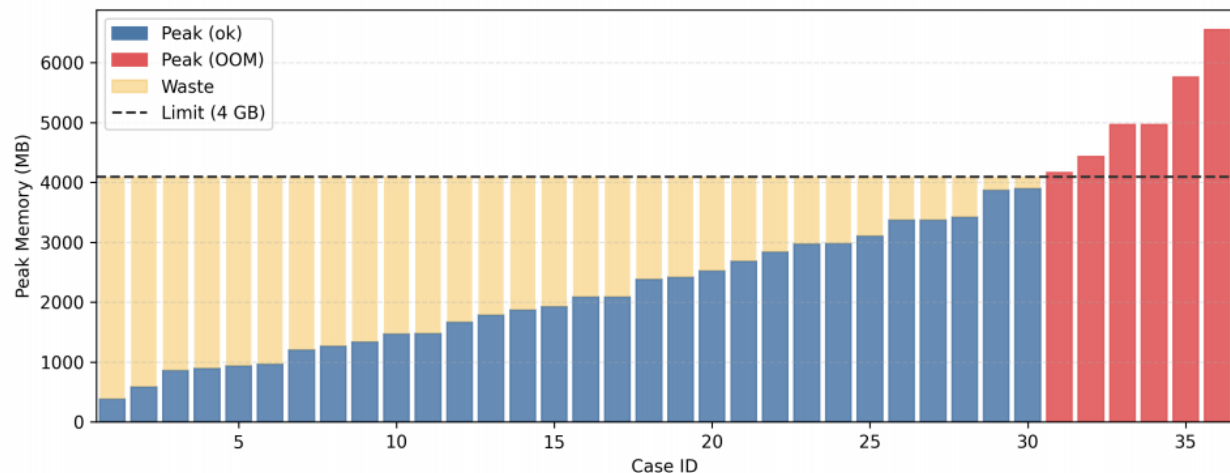
研究背景与动机



- **Problem:** 如何给无服务器计算函数分配合适的资源配置。
 - **Input:** 用户提交的无服务器函数源代码, 以及触发该函数的运行时输入数据 (如视频文件、图片)
 - **Output:** 每次函数调用时的最优资源配置 (如内存大小)
 - **Significance:** 防止资源的浪费, 避免oom导致任务失败的问题

研究背景与动机

静态配置（手动配置）



静态配置的固有困境

- 资源浪费；内存不足（OOM）任务执行失败
- 手动调优不现实（AWS Lambda 支持以 1MB 为增量设置 128MB 至 10240MB 的内存，超 10000 种内存档位）

自动化配置

贝叶斯优化（BO）
核心逻辑：多轮试错→拟合
概率模型→预测最优配置→
迭代收敛

无代
码语
义理
解的
黑盒
优化

强化学习（RL）
核心逻辑：环境交互→试错
调整策略→奖励反馈→训练
收敛

黑盒优化的固有困境

无法理解代码语义，需要大量的样本训练，收敛速度慢

研究背景与动机

- **Problem:** 如何给无服务器计算函数分配合适的资源配置。
 - **Input:** 用户提交的无服务器函数源代码，以及触发该函数的运行时输入数据（如视频文件、图片）
 - **Output:** 每次函数调用时的最优资源配置（如内存大小）
 - **Significance:** 防止资源的浪费，避免oom导致任务失败的问题
- **Limitations:**
 - **L1 (B1)** : 无法兼顾动态适配性与实时性，要么配置固定导致资源浪费和OOM任务失败，要么LLM 推理延迟过高无法生产
 - **L2 (A1-B2)** : 黑盒优化，需要大量样本，收敛速度慢

	A1: 无代码语义感知	A2: 具备代码语义感知
B1: 静态策略生成	A1-B1: 静态配置（手动配置）	A2-B1: LLM 在线生成 （请求一次LLM时间长， 链路延迟高）
B2: 动态进化策略	A1-B2: 贝叶斯优化/强化学习 （不理解语义，效率低）	A2-B2: Fuyun

研究背景与动机

研究机遇



技术前沿 • Frontier Tech

近年大语言模型（LLMs）与智能体框架在代码生成、代码理解，任务调度等复杂领域展现出卓越的处理性能。



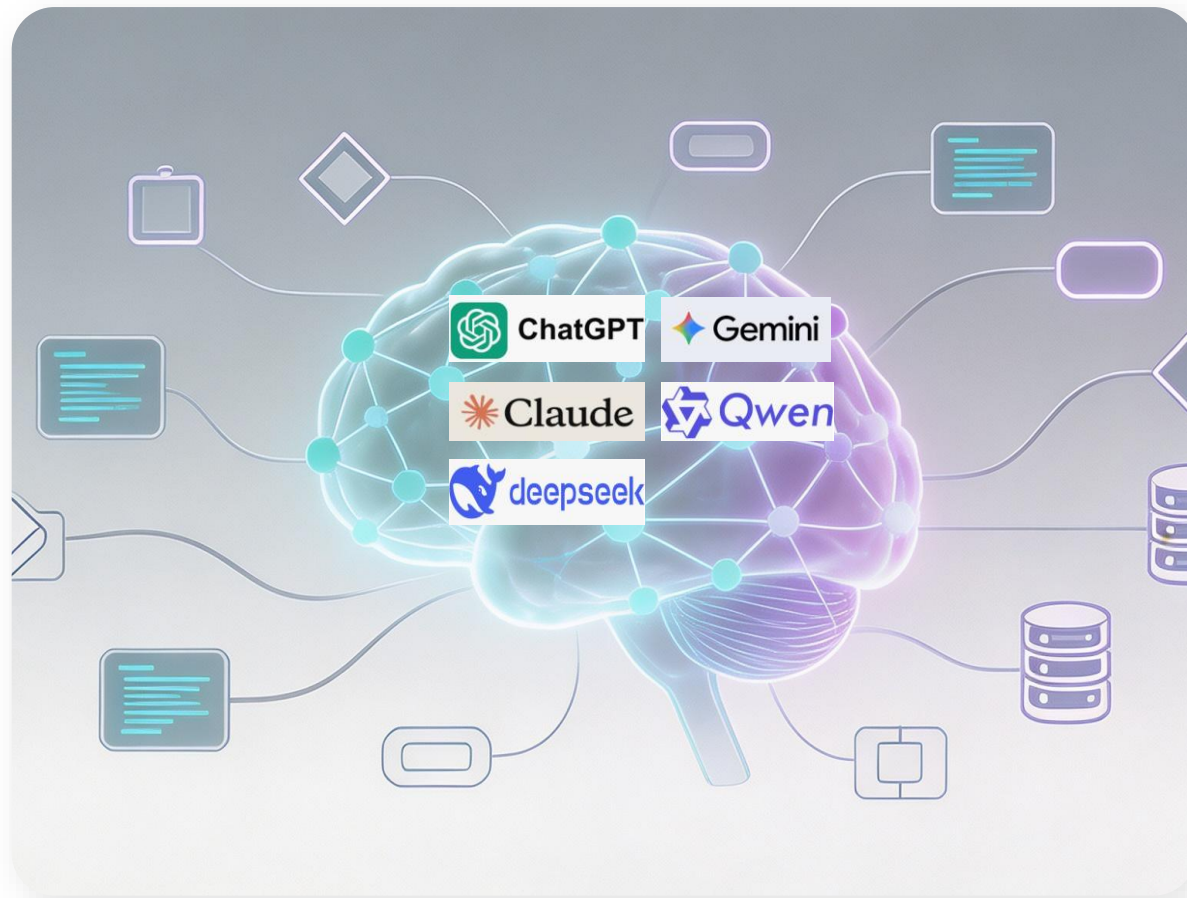
核心能力 • Core Ability

具备先进的语义理解与逻辑推理能力，能够深度解析代码底层逻辑，精准捕捉用户的任务真实意图。



研究机遇 • Research Opportunity

利用LLM的语义能力，从根本上解决无服务器资源配置中存在的“语义鸿沟”难题。



研究背景与动机

- **Problem:** 如何给无服务器计算函数分配合适的资源配置。
 - Input: 用户提交的无服务器函数源代码, 以及触发该函数的运行时输入数据 (如视频文件、图片)
 - Output: 每次函数调用时的最优资源配置 (如内存大小)
 - Significance: 防止资源的浪费, 避免oom导致任务失败的问题
- **Limitations:**
 - L1 (B1) : 无法兼顾动态适配性与实时性, 要么配置固定导致资源浪费和OOM任务失败, 要么LLM 推理延迟过高无法生产
 - L2 (A1-B2) : 黑盒优化, 需要大量样本, 收敛速度慢
- **Opportunity:**
 - 可以使用LLM从函数源代码中推导 “输入特征 - 资源消耗” 的内在因果关联, 给出关系表达式, 例如资源消耗= f (输入特征)



LLM集成的三大核心挑战



概率性与确定性的冲突

LLMs作为概率性生成器，其输出存在“幻觉”风险，可能产生不合规的值。直接将未经检查的输出映射到基础设施配置，会带来严重的操作风险。



高延迟与实时性的冲突

LLM推理的秒级延迟与无服务器函数毫秒级启动的严格实时需求存在根本性时间错配。同步查询LLM在生产环境中是性能上不可接受的。



静态预测与动态环境的冲突

仅基于源代码的静态预测无法解释运行时噪声或硬件异构性。若不进行持续的运行时适配与校准，将不可避免地导致预测精度的漂移。

研究背景与动机

- **Problem:** 如何给无服务器计算函数分配合适的资源配置。
 - Input: 用户提交的无服务器函数源代码, 以及触发该函数的运行时输入数据 (如视频文件、图片)
 - Output: 每次函数调用时的最优资源配置 (如内存大小)
 - Significance: 防止资源的浪费, 避免oom导致任务失败的问题
- **Limitations:**
 - L1 (B1) : 无法兼顾动态适配性与实时性, 要么配置固定导致资源浪费和OOM任务失败, 要么LLM 推理延迟过高无法生产
 - L2 (A1-B2) : 黑盒优化, 需要大量样本, 收敛速度慢
- **Opportunity:**
 - 可以使用LLM从函数源代码中推导 “输入特征 - 资源消耗” 的内在因果关联, 给出关系表达式, 例如资源消耗= f (输入特征)
- **Challenges:**
 - 概率性与确定性的冲突
 - 高延迟与实时性的冲突
 - 静态预测与动态环境的冲突
- **Method:**
 - 如何解决上述Challenges



作者介绍

研究背景与动机

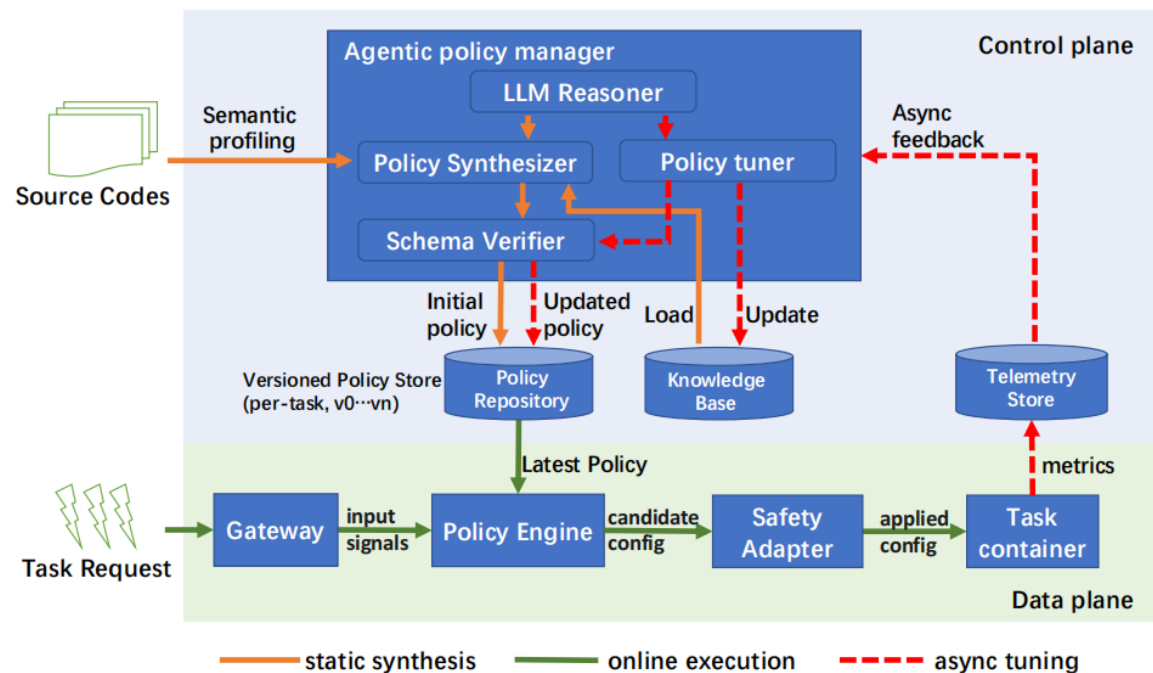
系统设计

实验评估

总结与讨论

系统设计

针对黑盒优化的语义盲、高样本依赖、生产不可用的核心瓶颈，Fuyun将资源配置从「数值统计猜测」重构为「符号策略合成任务」，首次用LLM智能体实现语义驱动的资源调度。



System Overview of Fuyun

1. Schema约束的符号策略合成->概率性与确定性的冲突
严格JSON格式规范约束LLM输出，从根源解决幻觉问题，生成可解释、可验证、可动态实例化的资源策略。
2. 离线和在线的双平面架构->高延迟与实时性的冲突
离线控制平面完成LLM策略合成，彻底规避LLM推理延迟；在线数据平面轻量执行策略，近零运行时开销。
3. 异步进化调优->静态预测与动态环境的冲突
基于运行时真实数据异步优化策略参数，仅需1/4的样本即可达到全可靠性，适配生产环境的严格约束。

研究背景与动机

- **Problem:** 如何给无服务器计算函数分配合适的资源配置。
 - Input: 用户提交的无服务器函数源代码, 以及触发该函数的运行时输入数据 (如视频文件、图片)
 - Output: 每次函数调用时的最优资源配置 (如内存大小)
 - Significance: 防止资源的浪费, 避免oom导致任务失败的问题
- **Limitations:**
 - L1 (B1) : 无法兼顾动态适配性与实时性, 要么配置固定导致资源浪费和OOM任务失败, 要么LLM 推理延迟过高无法生产
 - L2 (A1-B2) : 黑盒优化, 需要大量样本, 收敛速度慢
- **Opportunity:**
 - 可以使用LLM从函数源代码中推导 “输入特征 - 资源消耗” 的内在因果关联, 给出关系表达式, 例如资源消耗= f (输入特征)
- **Challenges:**
 - 概率性与确定性的冲突
 - 高延迟与实时性的冲突
 - 静态预测与动态环境的冲突
- **Method:**
 - Schema约束的符号策略合成
 - 离线和在线的双平面架构
 - 异步进化调优

该Schema通过标准化的结构，定义了策略的核心字段与逻辑，实现了运行时的高效实例化，并为策略的版本迭代与安全回滚提供了可靠的结构化支撑。

❖ 三大顶层核心组件

- Metadata**: 负责策略的唯一标识、版本控制与状态管理，实现全生命周期溯源。
- Inputs & Logic**: 声明可观测信号与符号变量映射，定义模型族、参数及决策逻辑。

🔄 统一异构输入表示

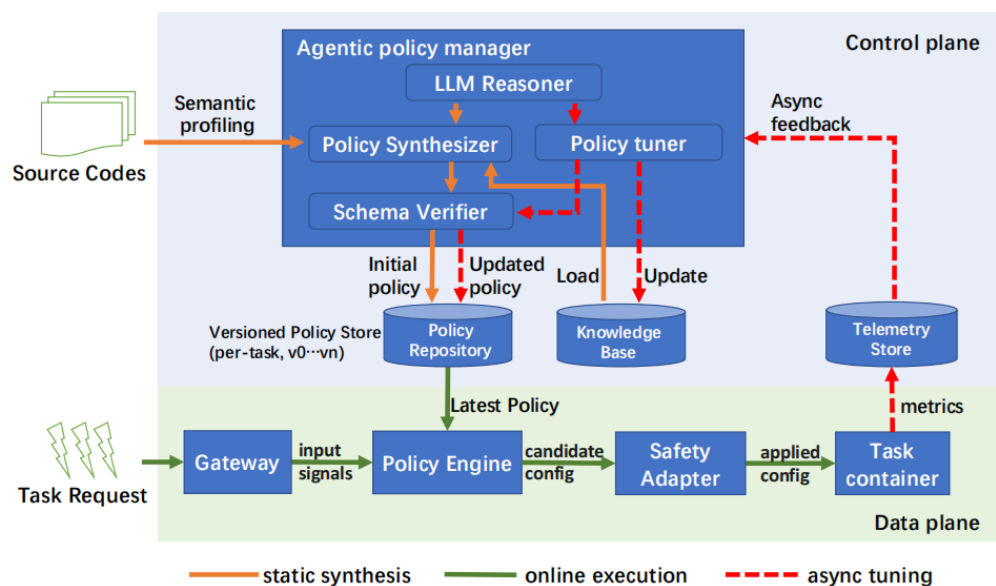
引入符号变量`x`作为运行时上下文的统一载体，通过确定性变换将异构的原始观测信号映射为标准的计算输入。

🧠 LLM 的智能生成角色

利用代码语义自动推导信号映射关系、选择最优模型族，并生成可审计的决策依据，确保策略逻辑的可复现性。

Section	Field	Purpose
Metadata	policy_id, task_id, version	识别每个任务的策略并支持版本化更新。
Metadata	status, supersedes	控制跨版本的激活和回滚。
Inputs	task_inputs.source	指定运行时上下文来源。
Inputs	variables	声明符号变量（包括x），并指定其类型和单位。
Inputs	mapping.x	定义所需的信号和计算x的确定性变换。
Logic	type	指定模型族。有效类型包括线性、分段线性、幂函数和指数函数。
Logic	coefficients	存储所选模型类型所需的可调参数。
Logic	decision_rationale	记录模型选择的理由和支持证据，以确保可审计性。

策略Schema生成流程详解



STEP 01 / 输入解析

输入任务源代码，提取核心逻辑文本



STEP 02 / LLM 语义分析

核心洞察：内存消耗与“视频总像素量”呈现极强的正相关性特征



STEP 03 / 信号选择与映射

信号选取：时长(Duration)、帧率(FPS)、分辨率

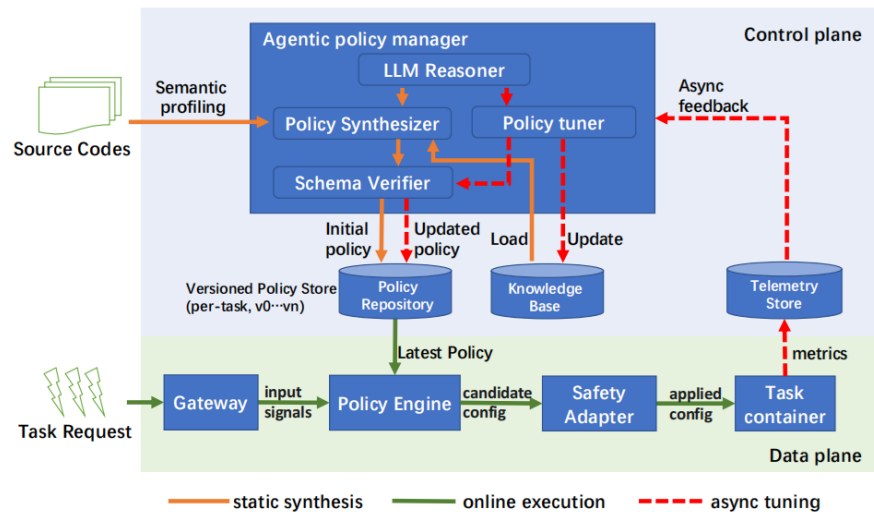
符号映射： $x = \text{duration} \times \text{fps} \times \text{width} \times \text{height} \times 3$

模型族选择： $y = \text{base} + \alpha \cdot x$



STEP 04 / 模型决策与记录

决策依据：记录模型选择的理由，确保策略可追溯。



Evolutionary Parameter Tuning

01

初始策略生成 (v0)

完全基于代码静态分析，不依赖历史数据。例如：为视频处理函数生成线性模型 $y = \text{base} + \alpha * x$ 。

02

在线调优触发条件

当监测到发生 OOM、预测误差超过阈值，或系统稳定运行一段时间后，自动触发调优流程。

03

动态参数更新机制

利用真实运行时数据，通过数值拟合方法（如最小二乘法、梯度下降）持续迭代更新模型中的参数（如 α 系数）。

04

模型结构动态重选

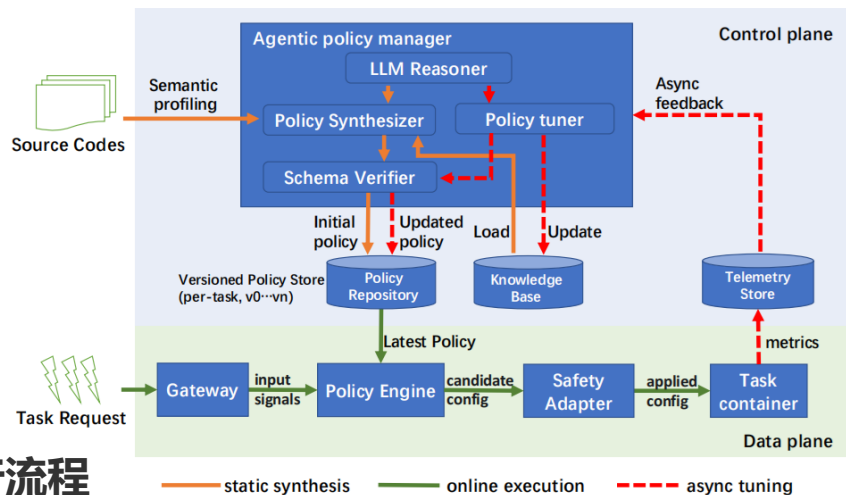
当参数微调无法解决精度问题时，系统会触发 LLM 重新介入，选择或生成更复杂的非线性模型结构。

异步调优核心逻辑

初始策略由 LLM 基于代码语义智能生成，为系统运行提供一个“合理的起点”。

随后，在线调优模块介入，利用真实运行时产生的监控数据，对初始策略进行持续的迭代与精调，最终将其收敛至最优状态。

这是一个从“粗粒度生成”到“细粒度调优”的持续进化过程。



适配执行流程

$$f_safe(r) = \text{SnapToTier} (r \times (1 + \delta_headroom)) \in [R_min, R_max]$$

增加安全余量：乘以系数 $(1 + \delta)$ 应对运行时瞬时资源尖峰，避免OOM。

映射离散档位：将连续计算值适配到平台支持的离散规格（如内存MB整数档位）。

限制配置边界：确保最终配置落在平台允许的 $[R_min, R_max]$ 物理限制范围内。

将策略引擎生成的候选资源配置 (r)，转换为云平台/硬件可执行的最终安全配置，解决运行时资源尖峰与平台离散配置限制问题。

资源适配计算链路

输入：策略候选值 (r)(来自策略引擎计算)

计算：增加安全余量 $(r \times (1 + \delta_headroom))$

转换：映射离散档位(SnapToTier / 取整适配)

输出：最终安全配置(Bound(R_min, R_max) / 合规落地)



作者介绍

研究背景与动机

系统设计

实验评估

总结与讨论



核心目标：验证 Fuyun 在资源配置任务中的收敛速度、可靠性与资源利用效率

01 研究问题 • Research Questions

- Fuyun能否快速收敛到稳定且准确的资源配置策略？
- 与传统静态配置及黑盒优化方法相比，其可靠性和效率表现如何？

02 基准测试 • Benchmark Suite

- **应用对象**：基于 SEBS 套件的视频处理函数 (Input-sensitive)。
- **核心特点**：内存消耗与输入视频的分辨率、时长等特征高度相关。

03 对比方法 • Baselines

- **静态配置**：固定内存限制 (2GB / 4GB / 8GB)。
- **GP-BO**：经典的黑盒优化算法 (贝叶斯优化)。

04 核心评估指标 • Metrics

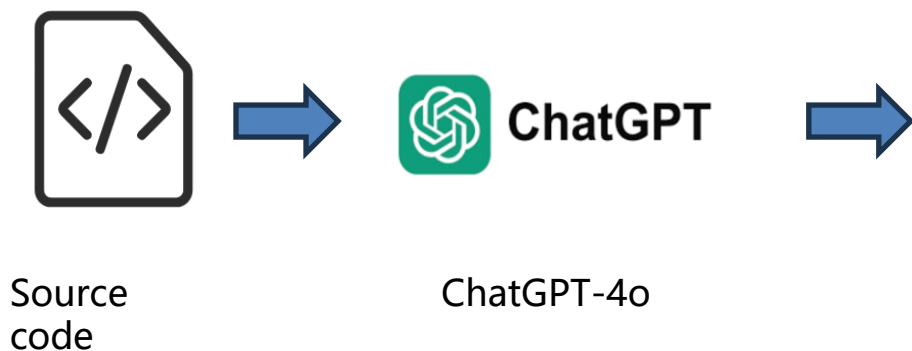
成功率 (Success Rate)

成功完成任务的比例，反映系统在不同输入下的**鲁棒性**与可用性。

浪费率 (Waste Ratio)

公式：(分配内存 - 实际峰值内存) / 分配内存
量化资源的过度分配情况，反映系统的**资源利用效率**。

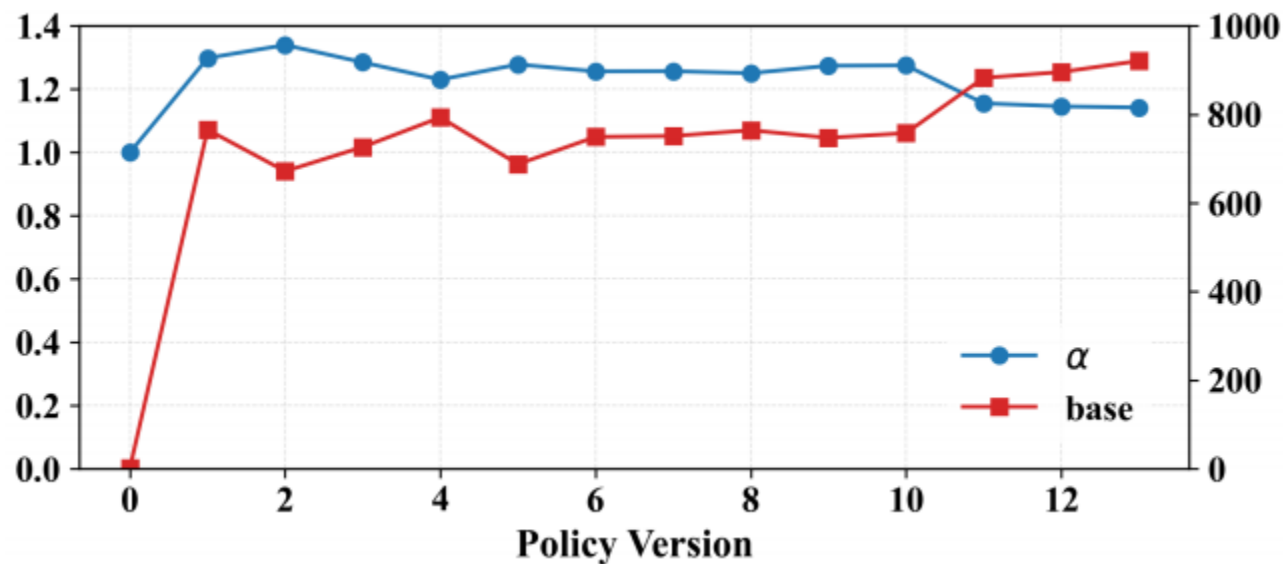
对于第一个问题，首先验证Fuyun是否能在无执行数据的情况下生成合适的初始策略。



大语言模型智能体通过分析视频处理函数源代码发现，内存占用量与视频尺寸及帧数呈现高度相关性。基于此推论，智能体采用线性模型近似内存占用量：

$$y = base + \alpha \cdot (\text{时长} \cdot \text{帧率} \cdot \text{宽度} \cdot \text{高度} \cdot 3)$$

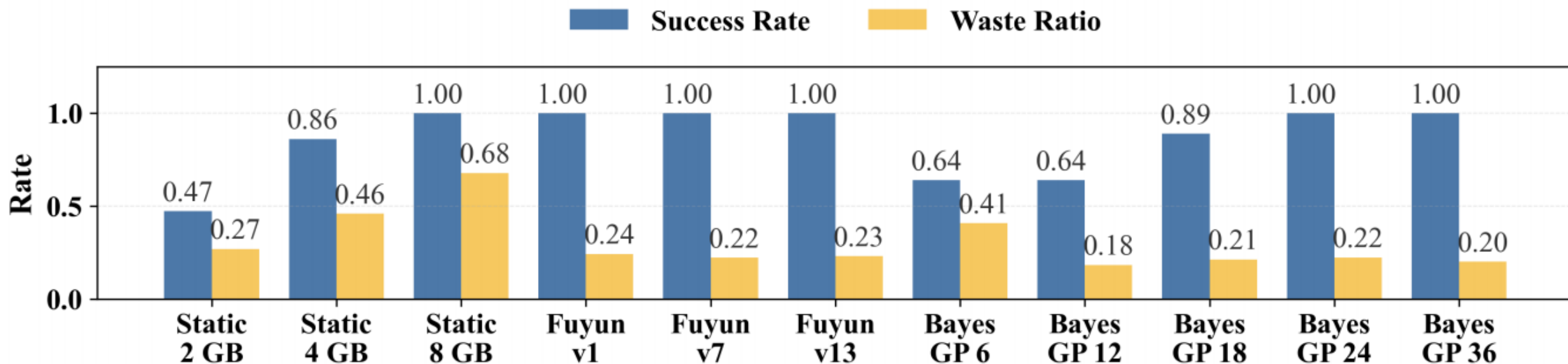
其中系数初始化为 $\alpha = 1.0$ 和 $base = 0$ 。这种理性推导方法相较于随机初始化提供了更优的起始点，有效缩小了后续动态阶段的搜索空间。



实验通过重放包含36次任务调用的轨迹数据来模拟生产场景。

在第一次参数拟合前，设置了 6 个样本的预热期。在预热期内，系统只收集真实运行产生的监控数据，不做任何参数调整、不更新策略版本。

图展示了线性模型参数 (α 和 base) 在调用序列中的收敛行为。如图所示，参数在预热期结束后立即进入快速调整阶段，有效修正了初始静态估计以匹配运行时实际状态。



vs. 静态配置 (2GB、4GB、8GB)

内存限制较低时，系统可靠性会显著下降

在100%成功率下，浪费率从0.68降至**0.23**，**降低了44%**的资源浪费，实现了极致的资源利用率。

vs. 贝叶斯优化 (GP-BO)

N是样本数

达到100%成功率仅需**6个**样本，相比GP-BO的24个样本，收敛速度提升了**3倍**(仅需1/4样本)。



作者介绍

研究背景与动机

系统设计

实验评估

总结与讨论

总结与讨论



一、能不能用到我们的场景

- 核心目标一致：都是基于 LLM 的智能资源调度配置，最终要实现负载感知的动态资源适配。黑盒优化（基于规则、强化学习）
(无服务器侧是「可靠性 - 成本 - 资源浪费」，手机端是「性能 - 功耗 - 续航 / 发热」)
- 待解决的共性问题高度重合：将 LLM 用于手机调度器配置，同样会遇到 Fuyun 解决的三大核心问题
 1. LLM 幻觉（生成超出手机硬件限制的 CPU/GPU 频率，导致死机 / 发热失控）
 2. LLM 推理延迟（手机端算力有限，LLM 实时推理与调度的低延迟需求冲突）
 3. 静态预测与动态负载不匹配（日常应用/游戏/视频等负载动态变化，纯 LLM 静态生成的配置无法适配实时负载）

二、能否泛化

问题：只要是能看清任务逻辑、输入特点、运行规律的资源调度场景，现在普遍都在“盲目调度”——要么靠固定配置，要么靠反复试错摸索，完全没用到任务本身的逻辑、输入和行为规律。结果就是资源给多了浪费、给少了崩，调度反应慢、试错成本高，既不稳定也不高效。

思考：把 Fuyun 的核心思路用到所有这类调度上。用大模型看懂任务的逻辑、输入和运行特点，生成清晰好用的调度规则；运行时直接按规则快速配资源；再根据实际运行情况不断微调规则。这样一来，所有能看懂语义的资源调度，都不用再靠固定配置和黑盒优化，既智能又稳定高效。

三、能不能进一步提高

问题：

Fuyun 的策略表达只有四种固定数学形式（线性、分段线性、幂、指数），且只依赖一个合并后的符号变量 x （比如 分辨率 $\times$ 帧率 $\times$ 时长 $\times 3$ ）。但真实资源消耗可能是多个因素独立作用再叠加（例如输入大小和并发数同时涨，资源不是简单相乘），也可能出现非单调变化（如内存先升后降）。固定单一公式无法表达这种复杂关系。

思考：

不让 LLM 只选公式模板，而是让它生成一段带条件判断的简易计算逻辑。比如：“如果并发数大于一定数量，则按并发数去计算；否则按输入大小算”。这样策略就变成了一个小型的决策流程，而不是一个固定等式。运行时先看当前条件走哪条分支，再计算资源。